

Surrogate-assisted neural learning and evolutionary optimization for expensive constrained multi-objective problems

Wenji Li^a, Yifeng Qiu^a, Zhaojun Wang^a, Biao Xu^a, Zhifeng Hao^b, Qingfu Zhang^c, Yun Li^d, Zhun Fan^{d,*}

^a College of Engineering, Shantou University, Shantou 515063, China

^b College of Science, Shantou University, Shantou 515063, China

^c Department of Computer Science, City University of Hong Kong, 999077, Hong Kong, China

^d Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518000, China

ARTICLE INFO

Keywords:

Expensive constrained multi-objective optimization
Neural learning-based solution generation
Surrogate-assisted optimization

ABSTRACT

Expensive constrained multi-objective optimization problems (ECMOPs) present significant challenges due to the high computational cost of evaluating objective and constraint functions, which severely limits the number of feasible function evaluations. To address this issue, we propose an efficient surrogate-assisted constrained multi-objective evolutionary algorithm, named LEMO. LEMO integrates neural learning with a novel constraint screening strategy to dynamically construct surrogate models for the most relevant constraints. During the optimization process, a neural network is designed to learn the mapping between arbitrary weight vectors and their corresponding constrained Pareto optimal solutions. This enables the generation of high-quality solutions while requiring fewer expensive function evaluations. Additionally, a constraint screening mechanism is introduced to dynamically exclude constraints that are irrelevant to the current search phase, thus simplifying the surrogate models and improving the efficiency of the constrained search process. To evaluate the effectiveness of LEMO, we compare its performance against seven state-of-the-art algorithms on three benchmark suites, LIRCMP, DASCMP, and MW, as well as a real-world optimization problem. The experimental results demonstrate that LEMO consistently outperforms these algorithms in both computational efficiency and solution quality.

1. Introduction

Many real-world optimization problems, such as neural architecture search [1], flow-shop group scheduling [2], and modular design automation of robots [3], can be categorized as constrained multi-objective optimization problems (CMOPs). Without loss of generality, a CMOP can be defined as follows:

$$\begin{cases} \min & \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x}))^T \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, p \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, q \\ & \mathbf{x} = (x_1, x_2, \dots, x_D)^T \in R^D \end{cases} \quad (1)$$

where $\mathbf{F}(\mathbf{x})$ is an M -dimensional objective vector, $\mathbf{x}$ is a D -dimensional decision vector within the decision space R^D . The function $g_i(\mathbf{x})$ denotes the i th inequality constraint, while $h_j(\mathbf{x})$ represents the j th equality constraint. It is worth noting that by relaxing constraints, such as setting the absolute value of an equality constraint $|h_j(\mathbf{x})|$ to be

less than a very small positive number ϵ , equality constraints can be converted into inequality constraints.

Compared to unconstrained multi-objective optimization problems (MOPs), CMOPs not only involve managing multiple conflicting objectives but also encounter additional challenges due to the presence of constraints. In such problems, only solutions that satisfy all constraints—referred to as feasible solutions—are considered valid. The feasibility of a solution is typically determined by evaluating its constraint violation (CV), which is defined as follows:

$$CV(\mathbf{x}) = \sum_{i=1}^p \max(g_i(\mathbf{x}), 0) + \sum_{j=1}^q \max(|h_j(\mathbf{x})| - \epsilon, 0) \quad (2)$$

where ϵ is a small positive number. If $CV(\mathbf{x}) = 0$, then $\mathbf{x}$ is a feasible solution; otherwise, it is an infeasible solution.

In many practical engineering applications, CMOPs often involve high computational costs, as accurately evaluating objectives and constraints demands significant computational resources and time. When

* Corresponding author.

E-mail address: fanzhun@uestc.edu.cn (Z. Fan).

such evaluations become particularly resource-intensive, these problems are classified as expensive constrained multi-objective optimization problems (ECMOPs).

To tackle ECMOPs, surrogate-assisted evolutionary algorithms (SAEAs) [4] have emerged as a widely used approach. These methods build surrogate models from a limited set of samples derived from expensive function evaluations, employing these models as cost-effective approximations to substitute for computationally intensive evaluations. Popular surrogate models include Kriging models [5], radial basis function (RBF) models [6], and random forests [7], among others. Although surrogate models reduce the need for numerous expensive function evaluations, their inherent prediction errors can impair the search process of SAEAs. Specifically, when constraint surrogate models misclassify feasible solutions as infeasible or infeasible solutions as feasible, they can misguide the search direction, causing the population of SAEAs to become trapped in local optima.

To tackle the precision limitations of surrogate models in ECMOPs, researchers have developed specialized constraint-handling techniques. For instance, Deb et al. [8] systematically classified and evaluated methods for constructing surrogate models of objectives and constraints, offering a detailed analysis of the strengths and weaknesses of various surrogate model management strategies. Based on this, Zhang et al. [9] integrated two such strategies by employing constraint surrogate models with varying granularity. When the population is distant from feasible regions, a coarser-grained model simplifies global constraint management; as the population nears feasible regions, a finer-grained model enhances local accuracy. This adaptive approach mitigates the misclassification errors of constraint surrogate models. To further improve the precision of local surrogate models, Song et al. [10] proposed an influential point-insensitive Kriging-based model that minimizes the impact of outlier samples while retaining valuable sample information. In a subsequent study [11], they analyzed the interplay between optimization directions of objectives and constraints, enabling the algorithm to dynamically alternate between constrained and unconstrained search modes. This adaptability reduces dependence on potentially inaccurate constraint surrogate models and limits their interference during optimization.

Beyond constraint handling, solving ECMOPs efficiently requires identifying high-quality solutions with a limited budget of expensive function evaluations, where selecting the most promising candidates poses a significant challenge. To address this, Tian et al. [12] trained a neural network classifier on surrogate-generated candidate solutions to predict quality relationships between pairs of individuals, reducing the chances of selecting suboptimal candidates for costly evaluations. Similarly, Zhang et al. [13] leveraged uncertainty information from Kriging models, introducing probabilities of feasibility and superiority as new optimization criteria, which established a probabilistic Pareto dominance mechanism to enhance solution comparisons. Meanwhile, Li et al. [14] segmented the objective space into multiple regions and applied tailored infill criteria based on the population state of each region, thereby improving the convergence, diversity, and feasibility performance of the SAEA.

In most prior studies, all constraints are treated equally, with efforts centered on overcoming the challenges of expensive constraints by either enhancing the accuracy of the surrogate model or reducing their use. However, constraints vary in their impact depending on the search stage. Wu et al. [15] recognized this and tackled a single, particularly challenging constraint during the intermediate phase of the algorithm, bridging the unconstrained and constrained stages with promising results. However, focusing solely on the most difficult constraint per iteration falls short when dealing with numerous or complex constraints. Thus, dynamically identifying the relevance of the constraint and building simpler surrogate models accordingly is the key to boosting SAEA performance in ECMOPs.

Additionally, while existing research emphasizes the selection of the most promising solutions for expensive function evaluations, it largely

overlooks methods to actively generate such solutions. Learning-based solution generators, as surveyed by Liu et al. [16], have shown the potential to improve search efficiency. However, these approaches are typically limited to inexpensive or non-constrained scenarios. Developing proactive techniques to create high-quality solutions with a limited expensive evaluations remains a critical, unresolved challenge in ECMOPs. In summary, existing SAEAs present several critical limitations that motivate the development of a new framework explicitly tailored to ECMOPs:

Firstly, most existing SAEAs predominantly focus on expensive unconstrained multi-objective optimization problems, while relatively few approaches have been specifically developed to handle ECMOPs effectively. Current constraint-handling techniques utilized by these algorithms generally assume equal importance for all constraints throughout the optimization process, either uniformly addressing all constraints or entirely neglecting them to expedite convergence [11]. This approach is problematic because practical optimization scenarios clearly demonstrate that constraints vary significantly in their influence on the algorithm's search process. Specifically, only constraints close to the CPF substantially impact algorithm performance, while others may be negligible [17].

Secondly, current SAEAs mainly prioritize developing efficient candidate solution selection strategies (known as infill criteria), with insufficient focus given to the effective generation of high-quality candidate solutions. This oversight leads to notable difficulties when handling ECMOPs characterized by high-dimensional decision spaces. Existing neural network-based methods typically employ binary classifiers for solution quality evaluation, often necessitating extensive real function evaluations without explicitly facilitating the direct generation of improved candidate solutions [12]. While some recent approaches leverage neural networks to map lower-quality solutions to higher-quality counterparts—thus indirectly improving solution quality via evolutionary operators—such methods primarily address non-expensive, large-scale optimization problems and do not adequately cater to ECMOPs [18].

Furthermore, although emerging research using neural networks, such as Pareto learning methods [19–21], successfully generates optimal solutions by learning mappings from weight vectors to corresponding Pareto-optimal solutions, these studies predominantly address non-expensive or unconstrained optimization scenarios. Consequently, research specifically targeting ECMOPs remains insufficiently explored [22].

To overcome these limitations, we introduce a surrogate-assisted neural learning and evolutionary multi-objective optimization method (LEMO) to address ECMOPs. The primary contributions of this work are summarized as follows:

1. We propose a constraint-filtering strategy that simplifies the construction of constraint surrogate models. Specifically, our approach excludes constraints with minimal influence on the current search, significantly reducing the negative impacts caused by surrogate model prediction inaccuracies.
2. We propose a novel unsupervised neural network-based candidate solution generator capable of directly producing high-quality solutions corresponding to any arbitrary weight vector in the objective space. This generator substantially enhances search efficiency, especially within the constraints of limited expensive function evaluations.
3. We integrate this neural network-based solution generation methodology into a surrogate-assisted evolutionary algorithm framework, resulting in the development of our proposed LEMO algorithm. Extensive empirical results demonstrate that LEMO

significantly accelerates population convergence, effectively mitigates the risk of premature convergence to local optima, and markedly improves the overall efficiency and effectiveness in solving ECMOPs.

The remainder of this work is structured as follows. Section 2 offers a concise overview of the preliminaries, including the Kriging model and the neural learning-based solution generation method. Section 3 details the proposed LEMO approach. Section 4 presents a comparative experimental analysis of the LEMO algorithm against seven state-of-the-art SAEAs. Finally, conclusions are provided in Section 5.

2. Preliminaries

2.1. Kriging models

We employ the Kriging model [23] to construct surrogate models for both objective and constraint functions in ECMOPs, enabling approximate function evaluations that reduce the need for expensive computations. The Kriging model is widely used in SAEAs due to its ability to provide both predictions and uncertainty estimates [24].

To build the Kriging model, we start with a set of input samples $X = \{x_1, x_2, \dots, x_n\}$ and their corresponding outputs $Y = \{y(x_1), y(x_2), \dots, y(x_n)\}$. The model assumes that the underlying function follows a Gaussian process, where the covariance between points reflects their similarity. The covariance vector c , which captures the relationship between a new point x and the training points X , is defined as:

$$c = \begin{bmatrix} \text{cov}(x_1, x) \\ \text{cov}(x_2, x) \\ \vdots \\ \text{cov}(x_n, x) \end{bmatrix} \quad (3)$$

Similarly, the covariance matrix C , representing the relationships among the training points, is given by:

$$C = \begin{bmatrix} \text{cov}(x_1, x_1) & \text{cov}(x_1, x_2) & \dots & \text{cov}(x_1, x_n) \\ \text{cov}(x_2, x_1) & \text{cov}(x_2, x_2) & \dots & \text{cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(x_n, x_1) & \text{cov}(x_n, x_2) & \dots & \text{cov}(x_n, x_n) \end{bmatrix} \quad (4)$$

Using these, the Kriging model predicts the output $\hat{y}(x)$ for a new input x as follows:

$$\hat{y}(x) = \hat{\mu} + c^T C^{-1} (Y - \mathbf{1}\hat{\mu}) \quad (5)$$

Here, $\hat{\mu}$ is the estimated mean of the output samples, calculated as:

$$\hat{\mu} = \frac{\mathbf{1}^T C^{-1} Y}{\mathbf{1}^T C^{-1} \mathbf{1}} \quad (6)$$

where $\mathbf{1}$ is an n -dimensional vector of ones. This prediction combines a global mean $\hat{\mu}$ with a local adjustment based on the covariance between x and the training points.

Beyond predictions, the Kriging model quantifies uncertainty through the predictive variance $s^2(x)$, which is crucial for assessing the reliability of the surrogate model. The variance is computed as:

$$s^2(x) = \hat{\sigma}^2 \left[1 - c^T C^{-1} c + \frac{(1 - c^T C^{-1} c)^2}{\mathbf{1}^T C^{-1} \mathbf{1}} \right] \quad (7)$$

where $\hat{\sigma}^2$ is the estimated process variance, calculated as:

$$\hat{\sigma}^2 = \frac{(Y - \mathbf{1}\hat{\mu})^T C^{-1} (Y - \mathbf{1}\hat{\mu})}{n} \quad (8)$$

This predictive variance $s^2(x)$ provides a quantitative measure of uncertainty for any input x . In the context of SAEAs, this uncertainty estimate plays a vital role in guiding the optimization process by helping to select candidate solutions for expensive function evaluations, balancing exploration of uncertain regions with exploitation of promising ones.

2.2. Neural learning-based solution generator

Neural learning-based solution generators have been shown to be an effective method for generating high-quality solutions in previous research [16]. These methods typically utilize supervised learning, where neural networks are trained using poor-performing solutions as inputs and high-performing solutions as output labels [25,26]. The goal is to develop a model that iteratively generates improved solutions, enhancing the optimization process.

For example, Zhan et al. [27] used pairs of parent individuals and their better-performing offspring as training samples, training a neural network to capture improvements from crossover and mutation. This enabled the population to converge more effectively after traditional evolutionary optimization. Similarly, Liu et al. [18] trained a neural network to map poor solutions to better ones along the same weight vector direction. They generated improved offspring by using parent individuals as inputs, followed by crossover and mutation, which accelerated population convergence. In a later study, Liu et al. [28] employed two multilayer perceptrons to guide the evolutionary search: one modeled the unconstrained search process, while the other prioritized feasibility. However, these supervised learning methods depend heavily on a large number of expensively evaluated solutions, posing a challenge for their scalability. To overcome this limitation, Lin et al. [19] employed an unsupervised learning strategy that allows the solution generator to approach the true Pareto solution set from various objective preference directions without requiring pre-constructed sample pairs. However, this method is primarily designed for unconstrained MOPs, limiting its applicability to constrained or expensive scenarios.

The research on neural learning-based solution generators has predominantly focused on unconstrained or non-expensive MOPs. Nevertheless, we believe these methods offer significant potential for addressing ECMOPs for the following reasons: (1) Abundant Training Data from Surrogate Models: Neural learning-based methods require extensive training data, which surrogate models can provide in large quantities without expensive function evaluations, thereby supporting effective model training. (2) Adaptability to Constraints: By integrating constraint-handling techniques, solution generators originally designed for unconstrained settings can be adapted to manage constraints, making them suitable for ECMOPs. (3) Enhanced Search Efficiency: A well-trained generator can directly produce numerous high-quality solutions, significantly improving search efficiency.

3. Proposed method

The main workflow of the proposed LEMO method is shown in Fig. 1. The process begins with the construction of surrogate models for the constraints and objective functions. Then, candidate solutions are generated using a combination of neural learning-based optimization and evolutionary optimization methods. An infill criterion is applied to select the most promising solutions for expensive function evaluations. The algorithm then checks whether the termination conditions are satisfied. If not, the process iterates; otherwise, all expensively evaluated solutions are output.

The pseudocode for the LEMO algorithm is provided in Algorithm 1. First, N candidate solutions are generated using Latin hypercube sampling, evaluated through expensive function evaluations, and then stored in the archive set A (see Line 1). In Line 2, the parameters of the solution generator network are initialized, with the detailed network architecture explained in Section 3.2. In the main loop of the algorithm, the proposed constraint-filtering strategy is first applied to remove constraints that are not relevant at the current stage. Next, surrogate models are constructed for each objective function and the filtered constraints (see Line 4). The details of surrogate model construction are described in Section 3.1. The algorithm then proceeds to surrogate-assisted learning optimization and evolutionary optimization.

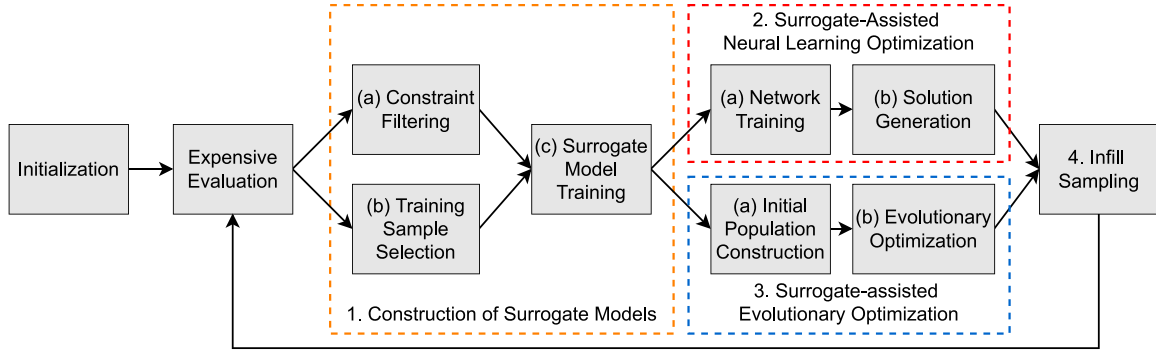


Fig. 1. Procedure of the proposed algorithm.

In the surrogate-assisted learning optimization process, a neural network is constructed to learn the mapping between arbitrary weight vectors in the objective space and their corresponding optimal solutions. This generates N high-quality solutions with different objective weight preferences, denoted as P_1 (see Lines 5–6). More details can be found in Section 3.2.

In the surrogate-assisted evolutionary optimization process, a region-based selection strategy is first used to select N well-converged and diverse solutions from the archive set A as the initial population. A constrained multi-objective optimization algorithm is then applied to evolve this population over $g_{\max}$ generations, generating the solution set P_2 (see Lines 7–11). Further details are provided in Section 3.3.

Next, the learning-generated population P_1 is merged with the evolution-generated population P_2 . The most promising solutions are selected through infill sampling (see Line 12), with the specific infill criterion described in Section 3.4. Finally, the selected candidate solution set P is evaluated using expensive function evaluations and added to the archive set A (see Lines 13–14). These steps (Lines 4–14) are iteratively repeated until the maximum number of expensive function evaluations is reached.

The following sections provide a detailed introduction to the processes involved in the construction of surrogate models, surrogate-assisted neural learning optimization, surrogate-assisted evolutionary optimization, and the infill sampling criterion within the LEMO algorithm.

3.1. Construction of surrogate models

We first apply a constraint-filtering strategy to remove redundant constraints, thus reducing approximation errors and computational complexity. Subsequently, we utilize a targeted training sample selection strategy to enhance surrogate accuracy around elite solutions. Separate Kriging models are then constructed individually for each objective and the active constraints.

3.1.1. Constraint-filtering strategy

To reduce the hindrance of constraints on the population's convergence process and facilitate the population's traversal of infeasible regions, we propose a novel constraint selection strategy based on the KTS algorithm [11]. This strategy analyzes the relationship between the convergence and feasibility of elite individuals, using the Pearson correlation coefficient to determine whether to conduct unconstrained or constrained searches. The convergence direction of elite individuals is evaluated using the convergence Q-metric [29], while feasibility is assessed using CV values.

In our strategy, we handle each constraint independently. By comparing the optimization direction of each constraint with the overall convergence direction of the population, we decide whether to consider that constraint. This approach avoids uniformly treating all constraints during the search process, enabling a more precise assessment of each

Algorithm 1: LEMO

Input: population size N , objective dimension M , decision dimension D

Output: archive A

```

1  $A \leftarrow$  generate  $N$  individuals using the Latin hypercube sampling method;
2  $NNmodel \leftarrow$  initialize an  $M$ -to- $D$  neural network;
3 while termination criterion is not met do
4    $Kmodel \leftarrow$  train Kriging models for objectives and constraints as described in Section 3.1;

   // Learning optimization as described in Section 3.2
5    $NNmodel \leftarrow$  iteratively train the  $NNmodel$ ;
6    $P_1 \leftarrow$  generate  $N$  solutions using  $NNmodel$ ;

   // Evolutionary optimization as described in Section 3.3
7    $P_2 \leftarrow$  select  $N$  individuals from  $A$  as the initial population;
8   for  $g \leftarrow 1$  to  $g_{\max}$  do
9      $O_2 \leftarrow$  generate  $N$  solutions from  $P_2$  using evolutionary reproduction;
10     $P_2 \leftarrow$  EnvironmentalSelection( $P_2 \cup O_2$ ,  $N$ );
11  end

  // Infill criterion as described in Section 3.4
12   $P \leftarrow$  InfillSampling( $P_1 \cup P_2$ );

13   $P \leftarrow$  apply expensive function evaluation to  $P$ ;
14   $A \leftarrow A \cup P$ ;
15 end
  
```

constraint's impact on convergence. Additionally, we disregard constraints that have been satisfied by all elite individuals, indicating that the population is largely within the feasible region defined by those constraints; thus, there is no need to consider them in the current iteration.

To accurately determine the direction of population optimization, we select the 20 individuals with the highest convergence metrics from the archive set A as elite individuals. At the start of each iteration, we first apply the previously described constraint-filtering method and then build separate surrogate models for each objective and the filtered constraints. This approach significantly reduces the time needed to construct surrogate models of constraints while alleviating the interference of the constraints with the search process. Calculating the Pearson correlation coefficient involves a single linear pass through the data

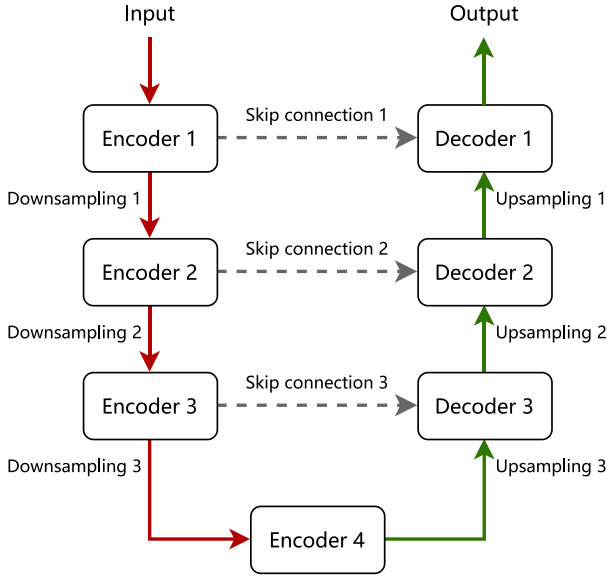


Fig. 2. General structure of U-Net.

points to calculate the mean, variance, and covariance, producing a time complexity of $O(n)$. In contrast, the Kriging model requires more intricate operations, such as constructing and inverting a covariance matrix, resulting in a time complexity of $O(n^3)$. Consequently, the constraint-filtering process consumes negligible time compared to the construction of the surrogate model.

3.1.2. Training sample selection

The training samples for the surrogate model consist of solutions that have undergone expensive function evaluations, all of which are stored in the archive set A . Samples located far from the Pareto front can compromise the surrogate model's accuracy near the Pareto front, consequently degrading the algorithm's performance. Furthermore, an insufficient sample size may lead to inadequate fitting of the surrogate model, while an overly large sample size increases the model's training time. Given that the algorithm's search primarily targets the vicinity of the current population, selecting high-quality solutions and their nearby counterparts as training samples ensures local accuracy of the surrogate model while simultaneously minimizing training time. For these reasons, our training samples are composed of two parts: high-quality samples and their neighboring samples.

To effectively balance the population's convergence and diversity, we employ a region-based selection strategy [30,31] that identifies high-quality samples by selecting solutions that span diverse objective preferences and lie close to the constrained Pareto front (CPF). The selection process proceeds as follows: First, N weight vectors are uniformly generated within the M -dimensional objective space. Next, each individual is assigned to the region associated with its nearest weight vector. Then, the fitness value of each individual is computed using the following method:

$$\psi(\mathbf{x}) = g_{\text{tche}}(\mathbf{x}|\mathbf{W}_x) + \alpha \cdot CV(\mathbf{x}) \quad (9)$$

where $\mathbf{W}_x$ is the weight vector associated with the region of individual $\mathbf{x}$, and g_{tche} is the Tchebycheff value, which evaluates the convergence of $\mathbf{x}$ along the direction of $\mathbf{W}_x$. Additionally, α is the penalty coefficient, and CV indicates the constraint violation of $\mathbf{x}$.

Next, we designate the individual with the lowest fitness value in each region as a high-quality sample. However, because some weight vectors may lack corresponding individuals, the total number of high-quality samples might fall below N after evaluating all regions. To

address this, we supplement the selection by choosing additional individuals from those not yet picked, prioritizing them in ascending order of fitness, until the total reaches N . After identifying the high-quality samples, we determine the three closest individuals in both the objective space and the decision space for each selected individual, forming their neighborhood sample points. We then combine the N high-quality samples with their respective six neighborhood samples, removing any duplicates to produce the training samples for the surrogate models. Finally, we construct a surrogate model for each objective function and each filtered constraint function.

3.2. Surrogate-assisted neural learning optimization

In this module, we design a U-shaped neural network that learns to map weight vectors to corresponding optimal solutions. We then adaptively sample weight vectors from the current solution archive to emphasize sparse regions, using these vectors as training data. This trained neural network subsequently generates new high-quality candidate solutions through iterative optimization, significantly enhancing search efficiency.

3.2.1. Neural network model

The solution generation method proposed in this study leverages weight vectors from the objective space to produce corresponding optimal solutions, effectively mapping the M -dimensional objective space to the D -dimensional decision space. As a versatile framework, it can incorporate any neural network model capable of translating M -dimensional inputs into D -dimensional outputs as the solution generator.

Since M is typically smaller than D , the input data often contains limited information, complicating the extraction of complex features. To tackle this challenge effectively, we adopt the U-Net [32] as the neural network model for the solution generator, with its general architecture depicted in Fig. 2. We implement the encoder, decoder, upsampling, and downsampling components using fully connected layers, as illustrated in Fig. 2. Specifically, our designed network model features M -dimensional inputs and D -dimensional outputs, with the number of neurons in the intermediate layers following the sequence of arrows in Fig. 2 as {32, 64, 64, 128, 128, 256, 256, 128, 128, 64, 64, 32}, corresponding to {Encoder 1, Downsampling 1, Encoder 2, Downsampling 2, ..., Upsampling 1, Decoder 1}.

3.2.2. Network training

To establish the mapping between weight vectors and constrained optimal solutions, we define the loss function as follows:

$$\mathcal{L}(\mathbf{W}) = \hat{g}_{\text{tche}}(\mathbf{x}|\mathbf{W}) + \lambda \cdot CV(\mathbf{x}), \quad \mathbf{x} = \text{Net}(\mathbf{W}) \quad (10)$$

Here, the function $\text{Net}()$ represents a neural network model that predicts the decision variables $\mathbf{x}$ corresponding to a given input weight vector $\mathbf{W} = (w_1, \dots, w_M)^T$. To assess network performance, we use the Tchebycheff value $\hat{g}_{\text{tche}}$ and the constraint violation $CV(\mathbf{x})$ derived from surrogate models, with λ serving as the penalty coefficient. The loss function incorporates both the Tchebycheff value and the constraint violation, enabling the balance between convergence and feasibility to be adjusted by tuning λ .

Given that the mapping between weight vectors and constrained optimal solutions is typically non-uniform, relying on uniformly selected weight samples for training can result in an uneven distribution of solutions across the objective space. To address this and improve the solution generator's ability to generate solutions in sparse regions while enhancing population diversity, we introduce an adaptive sampling strategy. This approach dynamically adjusts the sampling probability of each region according to the sparsity of the population distribution in the objective space.

Specifically, we begin by dividing the objective space into N regions, where N corresponds to the predefined population size. Next,

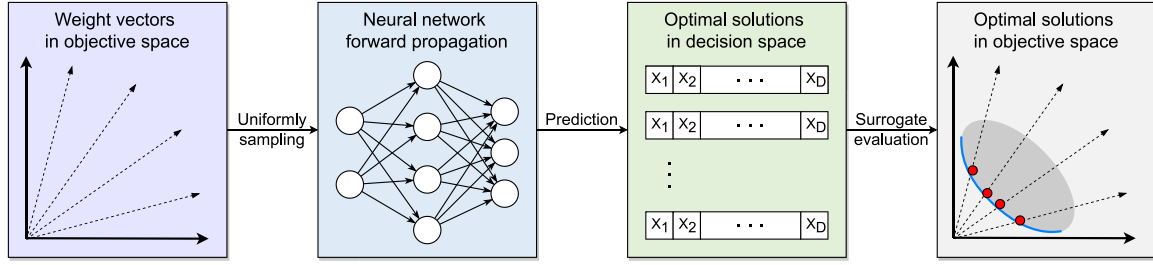


Fig. 3. Prediction process of the proposed solution generator.

we evaluate the number of existing solutions in each region and adjust the probability of selecting weight vectors from that region as training samples accordingly. Regions with fewer solutions are assigned higher sampling probabilities to prioritize them. For the i th region containing n_i solutions, we calculate its sampling score as $s(i) = \frac{1}{n_i + 1}$. The sampling probability for that region, $p(i)$, is then defined as follows:

$$p(i) = \frac{s(i)}{\sum_{i=1}^N s(i)} \quad (11)$$

In each training epoch, we first sample a batch of weight vectors from the objective space using the aforementioned method to serve as training samples. These samples undergo forward propagation through the neural network, after which the loss function is computed according to Eq. (10). Subsequently, the parameters of the network are updated through backward propagation. This process is repeated until the predefined maximum number of training epochs is reached.

3.2.3. Solution generation

The process of predicting high-quality solutions using the neural network is depicted in Fig. 3. After training the network model, we generate N weight vectors uniformly across the objective space as inputs, yielding N candidate solutions. These evenly distributed weight vectors span a range of objective preferences, allowing the network to produce a population with improved diversity. The crucial capability of the proposed neural learning optimization approach is progressively generating solutions of increasing quality throughout the optimization iterations. Experimental illustrations of the progressive improvements are supplemented in Section I-A of the appendix.

3.3. Surrogate-assisted evolutionary optimization

In the surrogate-assisted evolutionary optimization approach, we promote population diversity by initially employing a region-based selection strategy to choose N diverse solutions from the archive A as the starting population. This selection method mirrors the technique used to identify N high-quality training samples for the surrogate model, with specifics elaborated in Section 3.1. Next, we utilize a constrained multi-objective optimization algorithm to evolve this initial population across $g_{\max}$ generations, generating N candidate solutions. For this study, we select NSGA-II [33] as the optimizer. Notably, this flexible framework accommodates the integration of any constrained multi-objective optimization algorithm as the evolutionary optimizer.

3.4. Infill sampling

The goal of infill sampling is to identify a subset of promising solutions from the pool of candidate solutions for expensive function evaluation. Expected improvement (EI) [34–36] is a widely adopted infill criterion that simultaneously considers both the quality and uncertainty of solutions. To extend its utility to multi-objective scenarios, we integrate the individual fitness value from Eq. (9) with the EI metric, with the combined approach defined as follows:

$$EFI(x) = FI(x) \times \Phi\left(\frac{FI(x)}{s(x)}\right) + s(x) \times \phi\left(\frac{FI(x)}{s(x)}\right) \quad (12)$$

where Φ represents the cumulative distribution function, and ϕ denotes the probability density function. The term $s(x)$ reflects the uncertainty assessment of the solution x as estimated by the Kriging model. The fitness improvement value, $FI(x)$, is defined as follows:

$$FI(x) = \max(\psi_{\min} - \hat{\psi}(x), 0) \quad (13)$$

where $\psi_{\min}$ is the minimum fitness value obtained by the solutions in the archive.

In the infill sampling process, we begin by applying the K-means clustering method to divide the population into five clusters within the decision space. From each cluster, we then select the solution with the highest EFI metric for expensive function evaluation, yielding a total of five solutions from the candidate population. These evaluated solutions are subsequently incorporated into the archive set A .

4. Experimental study

To evaluate the effectiveness of the proposed LEMO algorithm, we conducted comparative experiments against seven state-of-the-art SAEAs. These evaluations were performed using three widely recognized benchmark suites—LIRCMOP [37], DASCOP [38], and MW [39]—alongside a real-world engineering case involving wellbore trajectory optimization [40]. Detailed definitions of the objective functions and constraints for the 37 benchmark problems, as well as the real-world engineering problem, are available in Refs. [37–40]. We evaluated the performance using three metrics: Inverted Generational Distance (IGD) [41], Hypervolume (HV) [42], and Success Rate (SR). IGD and HV together capture the algorithm's convergence and diversity, where smaller IGD values and larger HV values signify superior performance. To calculate IGD, we generated a set of 10,000 reference points; for HV, we fixed the reference points at (1.1, ..., 1.1). SR measures the proportion of runs yielding at least one feasible solution relative to the total number of runs. When an algorithm's SR falls below 100%, we use SR instead of IGD or HV to evaluate its performance. Additionally, we applied the Wilcoxon rank-sum test [43] with a 0.05 significance level to detect significant differences between pairs of algorithms. The symbols “+”, “−”, and “ $\approx$ ” denote that the compared algorithm performs significantly better than, worse than, or similarly to LEMO, respectively.

4.1. Experimental setup

We selected seven representative SAEAs for comparison with the proposed LEMO algorithm, including Multi-objective EGO [44], M1-2 [8], M2-2 [8], MGSAA [9], KTS [11], SA-CRVEA-AS [45], and SA-PPS [14]. The source code for all compared algorithms and benchmark problems is available via the PlatEMO [46] platform or by contacting their authors. The key parameters used in the experiments are summarized as follows:

1. The population size N for each algorithm is set to 100.
2. The maximum number of expensive function evaluations is set to 500.

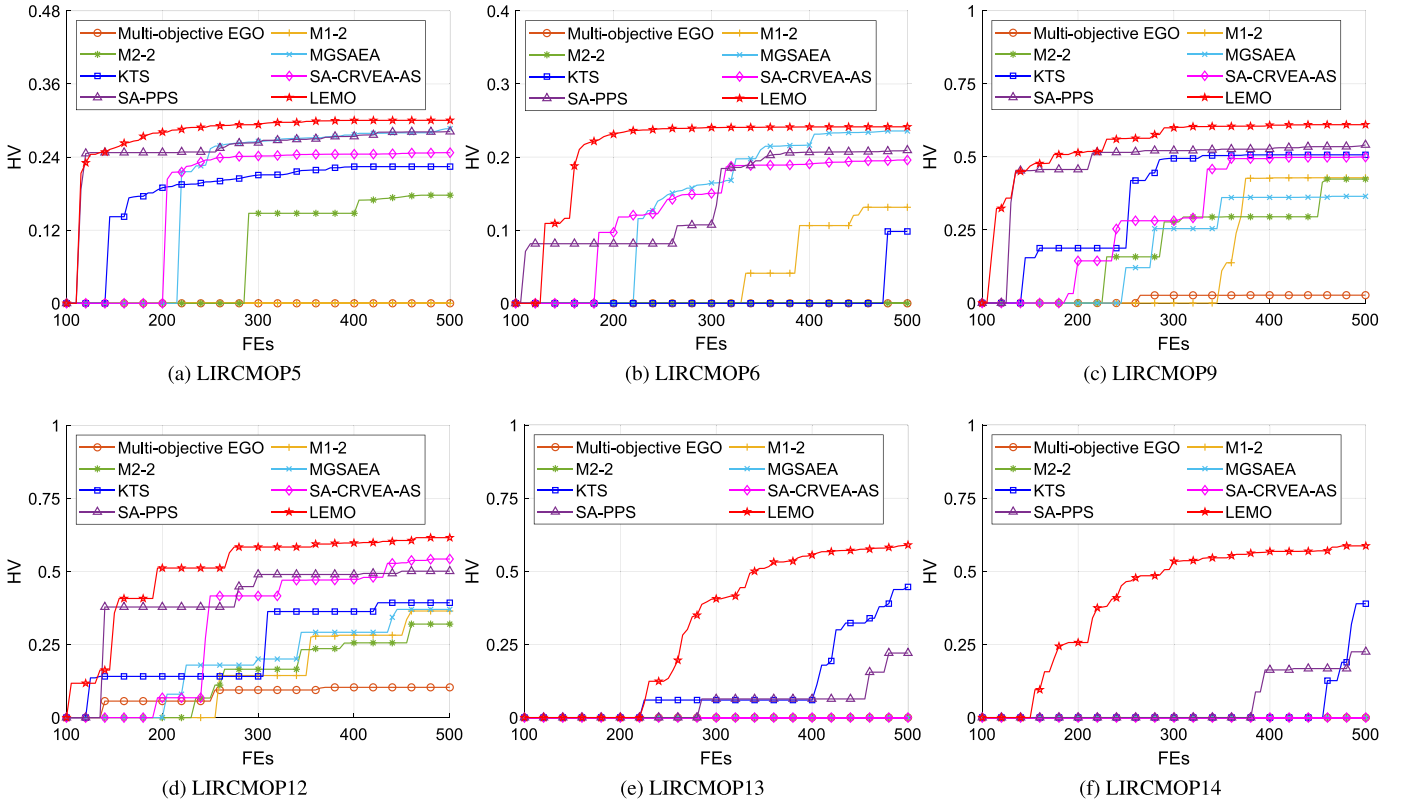


Fig. 4. HV change curve of population obtained by each algorithm on LIRCOP problem set.

Table 1

Wilcoxon rank-sum test results (+ /- / $\approx$) for the LEMO algorithm and the other seven ECMOEAs on the LIRCOP problem set.

Problem set	Metrics	Multi-objective EGO	M1-2	M2-2	MGSAA	KTS	SA-CRVEA-AS	SA-PPS	LEMO
LIRCOP	IGD	0/13/1	5/8/1	2/10/2	1/11/2	1/9/4	0/7/7	2/9/3	-
	HV	0/14/0	5/8/1	3/9/2	1/12/1	1/9/4	0/8/6	4/9/1	-

- The parameters of the comparison algorithms are set according to their original papers.
- For the LEMO algorithm, the maximum number of surrogate-assisted evolutionary generations per iteration, $g_{\max}$, is 20. The neural network training involves 200 epochs, a batch size of 32, and a learning rate of 0.005. The number of clusters for infill sampling is set to 5.
- The number of decision variables for the LIRCOP [37] and DASCOP [38] test sets is set to 15, while that for the MW [39] test set is set to 12. Other problem parameters are configured based on their respective original papers.

4.2. Comparison results on the LIRCOP test set

The LIRCOP test set presents challenges due to large infeasible regions, which hinder algorithms from converging to the CPF and make it difficult for them to traverse these areas. Table 1 provides a statistical comparison of the proposed LEMO algorithm against seven other algorithms using the IGD and HV metrics. Detailed results for each algorithm on the LIRCOP1-14 test problems are available in Tables A.1 and A.2 of Appendix.

For the IGD metric, the LEMO algorithm delivered the best performance on 8 out of 14 problems. It significantly outperformed Multi-objective EGO, M1-2, M2-2, MGSAA, KTS, SA-CRVEA-AS, and SA-PPS on 13, 8, 10, 11, 9, 7, and 9 problems, respectively. In terms of HV results, the LEMO algorithm achieved the best performance on 8 out of 14 problems, significantly surpassing the other seven compared algorithms on 14, 8, 9, 12, 9, 8, and 9 problems, respectively.

Fig. 4 illustrates the variation curves of median HV values for the LEMO algorithm compared to seven other algorithms when solving six different problems from the LIRCOP benchmark set: LIRCOP5, LIRCOP6, LIRCOP9, LIRCOP12, LIRCOP13, and LIRCOP14. The results clearly demonstrate that our LEMO algorithm exhibits rapid HV growth in the early stages and achieves the highest HV values in the later stages. This indicates that the LEMO algorithm significantly improves search efficiency, enabling it to quickly find high-quality solutions early on and further enhance population quality in subsequent iterations.

Fig. 5 illustrates the results for each algorithm based on the median HV for the LIRCOP10 problem. In this figure, the gray area indicates the feasible region, the blue lines denote the CPF, and the red dots represent the solutions obtained. The CPF of LIRCOP10 is fragmented by large infeasible regions, dividing it into several segments. This fragmentation poses a significant challenge for algorithms, as they need to navigate these infeasible regions and search within narrow, discrete feasible areas. To effectively illustrate the search results near the CPF for each algorithm, we employ the environmental selection strategy of NSGA-II to select the top N optimal solutions from the archive A for display when the final output solutions exceed N .

Fig. 5 shows that the LEMO algorithm successfully converges to all CPF segments isolated within narrow feasible regions, while the KTS, SA-CRVEA-AS, and SA-PPS algorithms manage to find only a few. The four algorithms, LEMO, SA-PPS, SA-CRVEA-AS, and KTS, demonstrate the ability to navigate through infeasible regions employing different adaptive strategies for constraint handling. For instance, SA-PPS ignores all constraints in the early search stage to traverse numerous

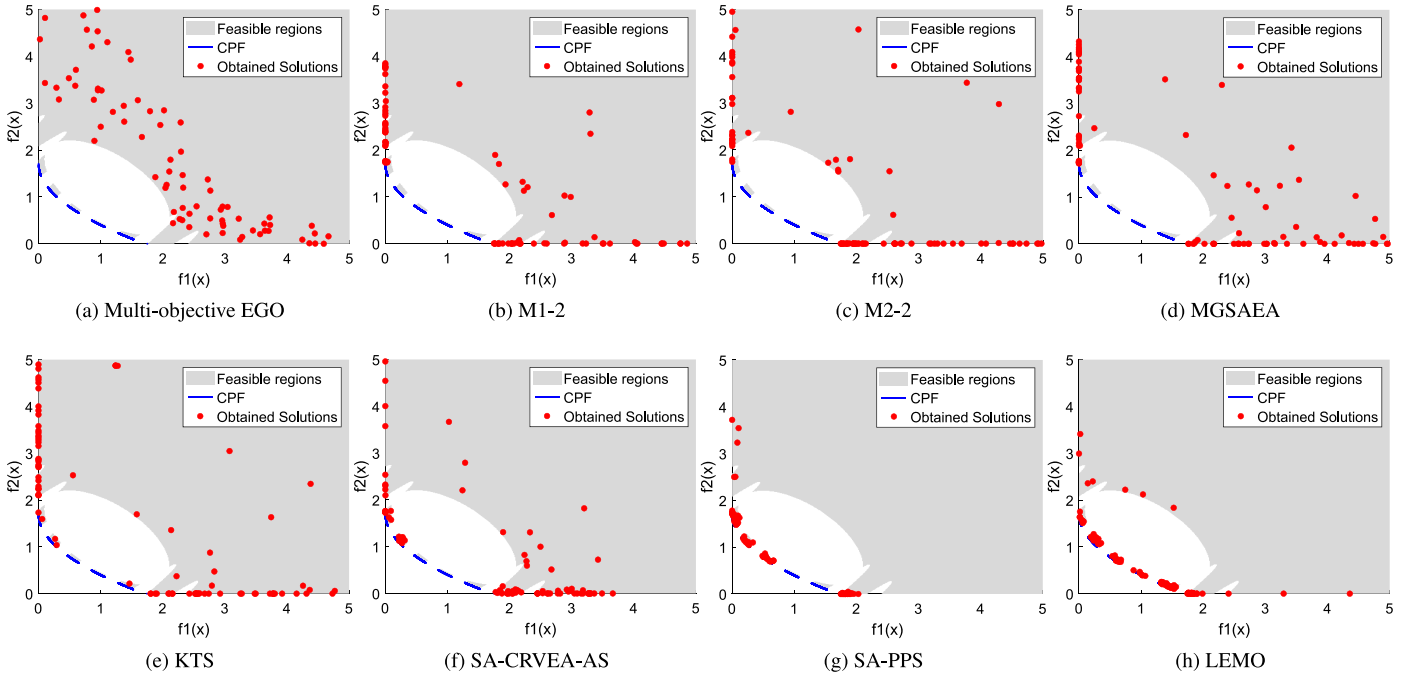


Fig. 5. Results achieved by each algorithm on LIRCOP10. The gray areas represent feasible regions, the blue lines denote the CPF, and the red dots signify the obtained N optimal solutions.

Table 2

Wilcoxon rank-sum test results (+ /- / $\approx$) for the LEMO algorithm and the other seven ECMOEAs on the DASCOP problem set.

Problem set	Metrics	Multi-objective EGO	M1-2	M2-2	MGSAAE	KTS	SA-CRVEA-AS	SA-PPS	LEMO
DASCOP	IGD	0/9/0	0/7/2	0/4/5	1/4/4	0/5/4	0/3/6	0/5/4	–
	HV	0/9/0	0/7/2	0/5/4	1/5/3	0/7/2	0/4/5	0/5/4	–

infeasible regions. When faced with obstacles generated by constraints, SA-CRVEA-AS and KTS switch to an unconstrained search mode to help their populations escape local optima. The LEMO algorithm goes further by not only refining constraint filtering but also employing neural learning-based methods to actively generate high-quality solutions, enhancing both diversity and convergence. In contrast, the populations of Multi-objective EGO, M1-2, M2-2, and MGSAAE are hindered by large infeasible regions, preventing them from reaching the CPF.

In summary, the experimental results demonstrate that the LEMO algorithm effectively addresses the challenges posed by large infeasible regions in the LIRCOP test set, such as extensive infeasible areas blocking access to the CPF and fragmentation of the CPF into discrete segments by infeasible regions, both of which significantly complicate the search process. Through the proposed constraint filtering strategy, LEMO adaptively determines which constraints to ignore, thereby significantly reducing the barriers posed by large infeasible areas and simplifying the search landscape. This capability helps overcome the common limitation of traditional algorithms, which often become trapped in local optima due to their inability to traverse infeasible regions. Additionally, compared to existing algorithms capable of crossing infeasible regions, our proposed learning-based optimization method achieves higher search efficiency. This efficiency enables early identification of high-quality solutions, improving population quality in subsequent generations.

4.3. Comparison results on the DASCOP test set

The DASCOP test set presents challenges due to a large number of constraints, leading to complex distributions of infeasible regions and irregular CPF shapes. Table 2 provides a statistical comparison of the proposed LEMO algorithm against seven other algorithms using the IGD and HV metrics. Detailed results for each algorithm on the

DASCOP1-9 test problems can be found in Tables A.3 and A.4 of Appendix.

According to the IGD results, the LEMO algorithm significantly outperformed Multi-objective EGO, M1-2, M2-2, MGSAAE, KTS, SA-CRVEA-AS, and SA-PPS on 9, 7, 4, 4, 5, 3 and 5 problems, respectively. For the HV metric, the LEMO algorithm significantly surpassed the other algorithms on 9, 7, 5, 5, 7, 4 and 5 problems, respectively.

Fig. 6 displays the variation curves of the median HV values for different algorithms when solving six different problems from the DASCOP benchmark set, specifically DASCOP1, DASCOP2, DASCOP3, DASCOP7, DASCOP8 and DASCOP9. From the figure, it can be observed that LEMO and SA-PPS achieve similar HV values in DASCOP1-3, but LEMO exhibits a faster HV growth rate during the early stages of the search. This indicates that our method can achieve efficient searches with fewer function evaluations. For DASCOP7-9, the LEMO algorithm achieves the highest HV values, significantly outperforming the other seven compared algorithms.

Fig. 7 illustrates the results for each algorithm based on the median HV for DASCOP1. In this figure, the gray region represents the feasible area, the blue lines denote the CPF, and the red dots indicate the obtained solutions. DASCOP1 features multiple complex infeasible regions, posing a significant challenge for constraint handling. To effectively visualize the search results near the CPF, we also used NSGA-II to select the optimal top N solutions of the archive for display when the final output exceeds N .

As depicted in Fig. 7, only the proposed LEMO and SA-PPS algorithms successfully converged to both CPF segments, overcoming the challenge posed by numerous complex constraints. Following them, the KTS and SA-CRVEA-AS algorithms each managed to locate one CPF segment. In contrast, Multi-objective EGO, M1-2, M2-2, and MGSAAE encountered difficulties with these intricate constraints, resulting in their populations failing to converge to the CPF.

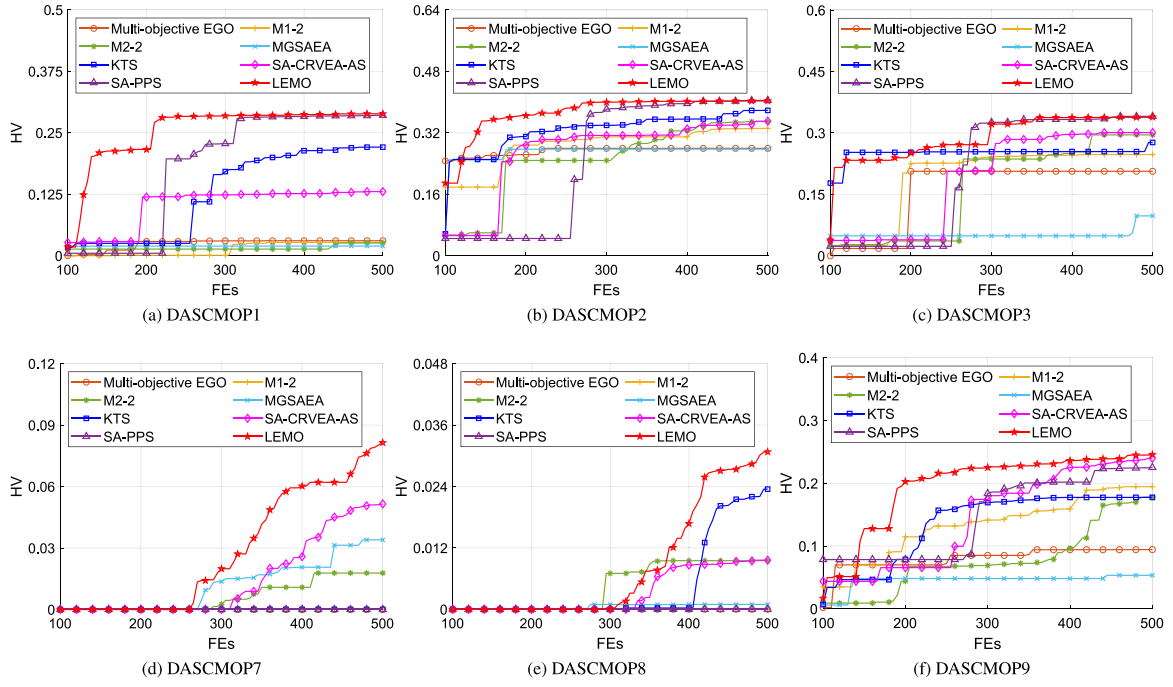


Fig. 6. HV change curve of population obtained by each algorithm on DASCOP problem set.

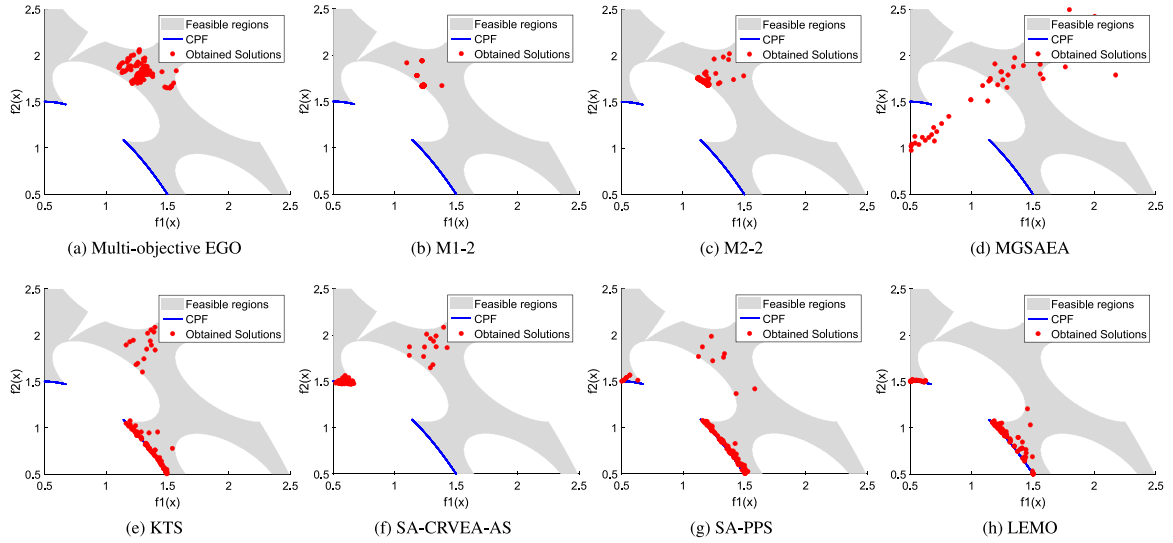


Fig. 7. Results achieved by each algorithm on DASCOP1. The gray areas represent feasible regions, the blue lines denote the CPF, and the red dots signify the obtained N optimal solutions.

In conclusion, these experimental results clearly illustrate that the LEMO algorithm maintains both strong diversity and convergence when tackling the DASCOP problem set, which includes numerous constraints. The key advantage arises from the proposed constraint filtering strategy, which evaluates constraints individually rather than collectively ignoring or considering them. This targeted filtering considerably reduces the complexity of the search space, while still preserving essential constraints to guide the algorithm's search direction. This advantage becomes more pronounced as the number of constraints increases, explaining LEMO's superior performance on the DASCOP problems.

4.4. Comparison results on the MW test set

The MW test set contains a large number of complex nonlinear constraints, which divide the CPF into multiple discrete segments and

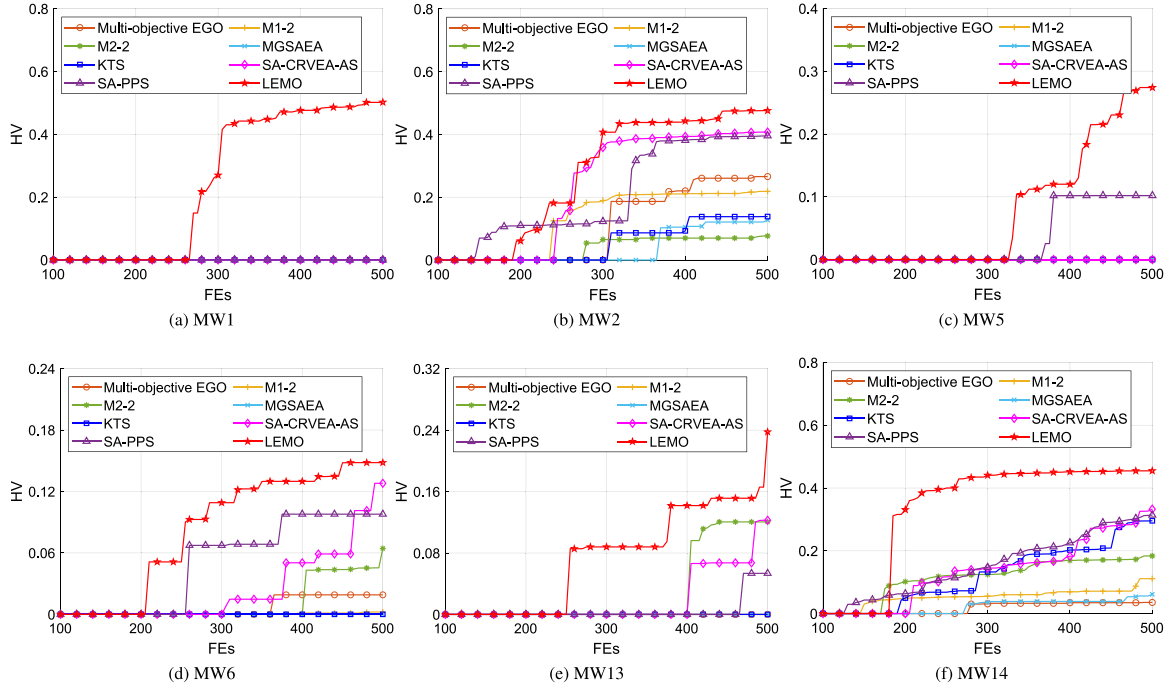
confine it to narrow and irregular feasible regions. Table 3 presents a statistical comparison of the proposed LEMO algorithm against seven other algorithms using the IGD and HV metrics. Detailed results for each algorithm on the MW1-14 test problems are available in Tables A.5 and A.6 of Appendix.

For the IGD metric, the LEMO algorithm achieved the best performance on 7 out of 14 problems. It significantly outperformed Multi-objective EGO, M1-2, M2-2, MGSAA, KTS, SA-CRVEA-AS, and SA-PPS on 13, 9, 10, 10, 12, 6, and 7 problems, respectively. In terms of HV results, LEMO achieved the best performance on 7 problems, significantly surpassing the other algorithms on 13, 10, 10, 10, 12, 7 and 6 problems, respectively.

Fig. 8 presents the variation curves of median HV values for different algorithms on six different problems from the MW benchmark set, specifically MW1, MW2, MW5, MW6, MW13, and MW14. The figure

Table 3Wilcoxon rank-sum test results (+ /- / $\approx$) for the LEMO algorithm and the other seven ECMOEAs on the MW problem set.

Problem set	Metrics	Multi-objective EGO	M1-2	M2-2	MGSAEA	KTS	SA-CRVEA-AS	SA-PPS	LEMO
MW	IGD	0/13/1	0/9/5	1/11/2	2/10/2	1/12/1	2/6/6	1/7/6	-
	HV	0/13/1	0/10/4	1/9/4	2/9/3	1/12/1	2/7/5	1/9/4	-

**Fig. 8.** HV change curve of population obtained by each algorithm on MW problem set.

clearly demonstrates that the LEMO algorithm consistently achieves the highest HV values throughout the search process. This highlights the superior efficiency of our method, which enables the discovery of higher-quality solutions with fewer expensive function evaluations.

In particular, for the MW1 problem, only the HV variation curve of the LEMO algorithm shows an increasing trend, while the HV values of the other seven comparison algorithms remain at 0. This is because only the LEMO algorithm achieved a success rate of 100%, while SA-PPS and MGSAEA achieved success rates of only 13.33% and 10%, respectively. The remaining five comparison algorithms had a success rate of 0%, failing to identify any feasible solutions across 30 independent runs. This further indicates that the proposed LEMO algorithm has strong exploration capability, effectively locating narrow feasible regions within large infeasible areas.

Fig. 9 displays the population results for each algorithm, based on the median HV values achieved for the MW14 problem. In this figure, the blue gridded surface depicts the CPF, while the red points indicate the solutions obtained. The CPF of MW14 comprises four irregular segments, presenting a significant challenge to the diversity of algorithms. The figure clearly reveals that the proposed LEMO algorithm and SA-PPS successfully target three of the four CPF segments, with LEMO demonstrating superior convergence as evidenced by the IGD metric. In contrast, Multi-objective EGO, MGSAEA, and SA-CRVEA-AS show potential to approach three CPF segments but remain considerable distant from the CPF. Meanwhile, M1-2, M2-2, and KTS exhibit limited diversity on this problem, each exploring only one of the four CPF segments.

In summary, the experimental results clearly indicate that the LEMO algorithm effectively addresses challenges posed by the MW problem set, characterized by generally narrow and discrete feasible regions. By leveraging the rapid convergence capability inherent in the proposed

learning-based optimization approach, LEMO efficiently identifies feasible regions containing the CPF. Specifically, the learning optimization method establishes explicit mapping relationships between arbitrary weight vectors in objective space and their corresponding optimal solutions, allowing direct and accurate prediction of high-quality solutions across diverse objective preferences. This predictive capability significantly enhances the algorithm's ability to locate discrete feasible regions. Additionally, when the population is located far from the CPF, the gradient-based optimization component within the neural learning-based solution generator efficiently guides population convergence along the path of steepest performance improvement, demonstrating considerable advantages for particularly challenging convergence problems, such as MW1.

4.5. Effectiveness of the neural learning-evolutionary cooperation

To validate the effectiveness of the collaboration between the learning optimization and the evolutionary optimization, we constructed two variants of the LEMO algorithm: LEMO-learn and LEMO-evo. In the LEMO-learn variant, only the neural learning-based method is used for the search, while in the LEMO-evo variant, the search relies solely on evolutionary optimization.

Table 4 presents the IGD results of the three algorithms on the LIRC-MOP benchmark problems, with the corresponding HV results provided in Table A.7 of Appendix. The LEMO algorithm demonstrates optimal performance on 6 out of 14 test problems, while its two variants excel on 4 problems each, showcasing the advantages of integrating learning and evolutionary approaches.

Notably, LEMO significantly outperforms LEMO-evo on 9 problems. However, the evolutionary variant performs better on three-objective problems (LIRC-MOP13-14). This performance difference is

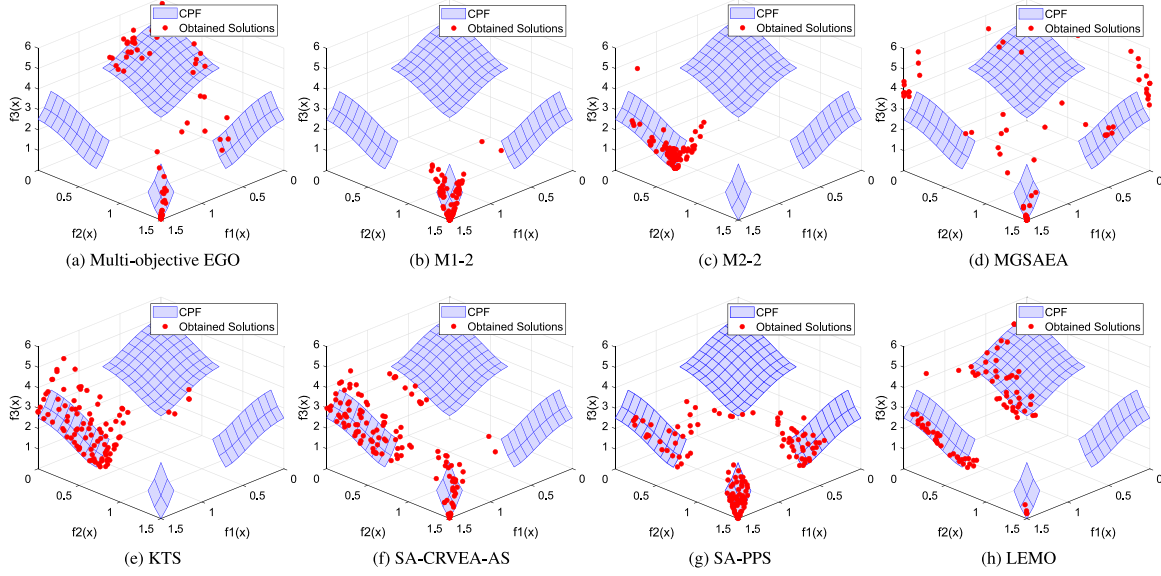


Fig. 9. Results achieved by each algorithm on MW14. The blue surfaces denote the CPF, and the red dots signify the obtained N optimal solutions.

Table 4

IGD results of LEMO and its two variants on LIRCMOP1-14. The terms '+', '-' and ' $\approx$ ' indicate that the result is significantly better, significantly worse or statistically similar to the results obtained by LEMO, respectively. The best result on each problem is highlighted in gray.

Problem	M	D	LEMO-evo	LEMO-learn	LEMO
LIRCMOP1	2	15	0% -	50% $\approx$	26.67%
LIRCMOP2	2	15	10% -	66.67% $\approx$	40%
LIRCMOP3	2	15	33.33% $\approx$	43.33% $\approx$	46.67%
LIRCMOP4	2	15	6.67% -	53.33% +	26.67%
LIRCMOP5	2	15	5.5372e-1 (5.12e-1) -	1.9418e-1 (1.11e-1) -	6.9221e-2 (3.31e-2)
LIRCMOP6	2	15	6.7278e-1 (4.03e-1) -	1.0946e-1 (7.74e-2) $\approx$	7.3440e-2 (4.25e-2)
LIRCMOP7	2	15	1.4205e-1 (3.81e-2) +	3.9554e+0 (3.09e+0) -	3.5105e-1 (2.08e-1)
LIRCMOP8	2	15	1.4844e-1 (5.10e-2) +	3.5201e+0 (3.29e+0) -	3.0626e-1 (1.49e-1)
LIRCMOP9	2	15	4.7139e-1 (5.51e-2) -	1.4085e-1 (1.12e-1) $\approx$	1.7635e-1 (1.73e-1)
LIRCMOP10	2	15	4.1295e-1 (9.45e-2) -	7.5252e-2 (4.34e-2) -	2.9985e-2 (7.15e-3)
LIRCMOP11	2	15	3.5429e-1 (1.16e-1) -	2.6476e-1 (9.69e-2) -	2.1191e-1 (8.53e-2)
LIRCMOP12	2	15	2.6981e-1 (5.36e-2) -	3.5544e-1 (2.96e-1) -	1.7217e-1 (5.99e-2)
LIRCMOP13	3	15	2.3597e-1 (5.16e-2) $\approx$	2.7967e-1 (1.12e-1) $\approx$	2.4707e-1 (2.40e-2)
LIRCMOP14	3	15	2.7479e-1 (1.24e-1) +	4.3238e-1 (1.56e-1) -	2.9164e-1 (2.47e-2)
+/-/ $\approx$			3/9/2	1/7/6	

likely due to the increased complexity of the loss function associated with higher objective dimensions, which can reduce the effectiveness of the learning-based optimization component in multi-objective problems.

Conversely, LEMO-learn achieves higher success rates on problems with extremely narrow feasible regions (LIRCMOP1-4). This is because the gradient-based optimization approach employed by LEMO-learn is particularly effective for precise local search in constrained spaces. In these cases, the ability to exploit local gradients allows LEMO-learn to navigate the search space more efficiently, especially when feasible regions are very tight.

Although the integration of learning and evolutionary optimization requires computational resources to be allocated between both components, which may result in slightly inferior performance on certain problems compared to using either method alone, the combined approach demonstrates enhanced robustness across most test cases. This improves the overall generalizability of the algorithm, enabling it to adapt effectively to a wide range of problem configurations.

In summary, the strength of our approach lies in the complementary nature of the learning and evolutionary components. While the evolutionary algorithm excels at exploring larger search spaces, the learning-based component enhances performance on problems with constrained regions, leading to more effective solutions across a diverse set of test cases.

4.6. Effectiveness of the constraint filtration strategy

To verify the effectiveness of the proposed constraint filtering strategy, we designed two variants of the LEMO algorithm: LEMO-Cons and LEMO-TS. The LEMO-Cons variant considers all constraints in every iteration, while the LEMO-TS variant adopts the constraint-handling technique of the KTS algorithm.

Table 5 presents the IGD results of the three algorithms, with the corresponding HV results provided in Table A.8 of Appendix. The LEMO algorithm achieved the best performance in 8 out of 14 problems, while each of the two variants performed best in 3 problems. Statistically, LEMO significantly outperformed LEMO-Cons on 8 problems. This indicates that employing an appropriate constraint-handling mechanism, which reduces the complexity of constraint surrogate models, significantly enhances the algorithm's performance. Additionally, LEMO significantly outperformed LEMO-TS on 5 problems and showed similar performance on 8 problems, suggesting that our proposed constraint filtering strategy is effective.

4.7. Effectiveness of training sample selection for surrogate models

Due to space constraints, the effectiveness of the training sample selection for surrogate models is discussed in Section I-H of Appendix.

Table 5

IGD results of LEMO and its two variants on LIRCMOP1-14. The terms '+', '-' and '≈' indicate that the result is significantly better, significantly worse or statistically similar to the results obtained by LEMO, respectively. The best result on each problem is highlighted in gray.

Problem	M	D	LEMO-Cons	LEMO-TS	LEMO
LIRCMOP1	2	15	40% ≈	20% ≈	26.67%
LIRCMOP2	2	15	36.67% ≈	10% –	40.00%
LIRCMOP3	2	15	20% ≈	16.67% –	46.67%
LIRCMOP4	2	15	13.33% ≈	3.33% –	26.67%
LIRCMOP5	2	15	2.7303e-1 (3.94e-1) –	7.4138e-2 (5.00e-2) ≈	6.9221e-2 (3.31e-2)
LIRCMOP6	2	15	2.8519e-1 (2.11e-1) –	6.1123e-2 (3.70e-2) ≈	7.3440e-2 (4.25e-2)
LIRCMOP7	2	15	2.1729e-1 (4.35e-2) +	3.6043e-1 (1.59e-1) ≈	3.5105e-1 (2.08e-1)
LIRCMOP8	2	15	2.4807e-1 (8.19e-2) ≈	4.8709e-1 (3.31e-1) –	3.0626e-1 (1.49e-1)
LIRCMOP9	2	15	3.9046e-1 (8.13e-2) –	1.7852e-1 (1.38e-1) ≈	1.7635e-1 (1.73e-1)
LIRCMOP10	2	15	1.7080e-1 (9.31e-2) –	3.6095e-2 (1.37e-2) ≈	2.9985e-2 (7.15e-3)
LIRCMOP11	2	15	3.4914e-1 (1.54e-1) –	2.1161e-1 (9.08e-2) ≈	2.1191e-1 (8.53e-2)
LIRCMOP12	2	15	3.0584e-1 (1.23e-1) –	2.1376e-1 (5.79e-2) –	1.7217e-1 (5.99e-2)
LIRCMOP13	3	15	9.6705e-1 (1.20e+0) –	2.5199e-1 (2.72e-2) ≈	2.4707e-1 (2.40e-2)
LIRCMOP14	3	15	6.9966e-1 (5.44e-1) –	2.7659e-1 (3.74e-2) +	2.9164e-1 (2.47e-2)
+/-/≈			1/8/5	1/5/8	

Table 6

HV results of LEMO and other seven algorithms on the wellbore trajectory optimization problem. The terms '+', '-' and '≈' indicate that the result is significantly better, significantly worse or statistically similar to the results obtained by LEMO, respectively. The best result on each problem is highlighted in gray.

Problem	M	D	Multi-objective EGO	M1-2	M2-2	MGSAEA	KTS	SA-CRVEA-AS	SA-PPS	LEMO
Wellbore trajectory	3	11	13.33% –	1.9088e-2 (4.95e-3) –	2.4242e-2 (8.50e-3) =	0% –	43.33% –	86.67% –	40% –	2.7816e-2 (2.92e-3)
+/-/=			0/1/0	0/1/0	0/0/1	0/1/0	0/1/0	0/1/0	0/1/0	

4.8. Parameter sensitivity analysis

Due to space constraints, the parameter sensitivity experiments are discussed in Section I-I, I-J and I-K of Appendix.

4.9. Real-world application

To further validate the practical effectiveness of LEMO in engineering applications, we applied it to the wellbore trajectory optimization problem [40]. The proposed LEMO algorithm was evaluated against seven other SAEAs on this problem, which features three objectives, ten decision variables, and multiple constraints. Additional details about the problem formulation are available in Section I-L of Appendix.

Given that the true CPF of the wellbore trajectory optimization problem is unknown, we employed the HV metric to assess each algorithm's performance. The maximum number of expensive function evaluations was limited to 1000, with other experimental settings aligned with those outlined in Section 4.1. Table 6 summarizes the HV results for each algorithm on this problem. Among them, only M1-2, M2-2, and LEMO consistently identified feasible solutions across all 30 runs, with LEMO outperforming the others, thus highlighting the effectiveness and superiority of the proposed LEMO algorithm in addressing real-world optimization challenges. Comprehensive experimental results are detailed in Section I-L of Appendix.

5. Conclusion

To address the challenge of limited evaluation budgets in ECMOPs, this paper presents a new unsupervised neural learning-based method for generating solutions. This method uses neural networks to connect weight vectors in the objective space to their optimal solutions, producing high-quality outcomes for diverse preferences. Building on this, we developed a surrogate-assisted neural learning and evolutionary optimization algorithm, named LEMO, for ECMOPs. In the learning phase, our solution generator creates high-quality solutions to speed up convergence. In the evolutionary phase, NSGA-II optimizes the search, expanding its scope and avoiding local optima. We also introduce a constraint filtering strategy that skips irrelevant constraints, reducing search obstacles and errors from constraint surrogates. To validate the effectiveness of the proposed method, we compared LEMO against

seven SAEAs on three benchmark test suites and a wellbore trajectory optimization problem. The experimental results demonstrate that our algorithm handles complex constraints more effectively within a limited number of expensive function evaluations, delivering solutions with strong convergence, diversity, and feasibility.

It is worth noting that LEMO is a highly extensible method for addressing ECMOPs. Both its neural learning-based solution generator and surrogate-assisted evolutionary optimization can be tailored to specific problems. In the future, we plan to explore more efficient constraint sampling techniques and more efficient neural learning-based solution generation methods to enhance the potential of the LEMO approach in solving ECMOPs.

CRedit authorship contribution statement

Wenji Li: Writing – review & editing, Writing – original draft, Methodology, Funding acquisition. **Yifeng Qiu:** Writing – review & editing, Writing – original draft, Methodology. **Zhaojun Wang:** Writing – review & editing, Writing – original draft. **Biao Xu:** Supervision, Conceptualization. **Zhifeng Hao:** Supervision, Funding acquisition. **Qingfu Zhang:** Methodology, Conceptualization. **Yun Li:** Supervision, Conceptualization. **Zhun Fan:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported in part by the National Science and Technology Major Project (grant number 2021ZD0111502), the National Natural Science Foundation of China (grant numbers 62176147, 62476163, 62441612), the Science and Technology Planning Project of Guangdong Province of China (grant numbers 2022A1515110660, 2021JC06X549), the Science and Technology Special Funds Project of Guangdong Province of China (grant numbers STKJ2021176, STKJ2021 019), the Guangdong Basic and Applied Basic Research Foundation (2023B1515120020, 2024A1515012450), and the STU Scientific Research Foundation for Talents (grant numbers NTF21001, NTF22030).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.swevo.2025.102020>.

Data availability

Data will be made available on request.

References

- [1] X. Zhou, Z. Wang, L. Feng, S. Liu, K.-C. Wong, K.C. Tan, Toward evolutionary multitask convolutional neural architecture search, *IEEE Trans. Evol. Comput.* 28 (3) (2024) 682–695.
- [2] X. He, Q.-K. Pan, L. Gao, L. Wang, P.N. Suganthan, A greedy cooperative co-evolutionary algorithm with problem-specific knowledge for multiobjective flowshop group scheduling problems, *IEEE Trans. Evol. Comput.* 27 (3) (2023) 430–444.
- [3] W. Li, Z. Wang, R. Mai, P. Ren, Q. Zhang, Y. Zhou, N. Xu, J. Zhuang, B. Xin, L. Gao, Modular design automation of the morphologies, controllers, and vision systems for intelligent robots: A survey, *Vis. Intell.* 1 (1) (2023) 2.
- [4] J.-Y. Li, Z.-H. Zhan, J. Zhang, Evolutionary computation for expensive optimization: A survey, *Mach. Intell. Res.* 19 (1) (2022) 3–23.
- [5] D. Zhan, H. Xing, A fast kriging-assisted evolutionary algorithm based on incremental learning, *IEEE Trans. Evol. Comput.* 25 (5) (2021) 941–955.
- [6] G. Li, Q. Zhang, Q. Lin, W. Gao, A three-level radial basis function method for expensive optimization, *IEEE Trans. Cybern.* 52 (7) (2022) 5720–5731.
- [7] H. Wang, Y. Jin, A random forest-assisted evolutionary algorithm for data-driven constrained multiobjective combinatorial optimization of trauma systems, *IEEE Trans. Cybern.* 50 (2) (2020) 536–549.
- [8] K. Deb, R. Hussein, P.C. Roy, G. Toscano-Pulido, A taxonomy for metamodeling frameworks for evolutionary multiobjective optimization, *IEEE Trans. Evol. Comput.* 23 (1) (2019) 104–116.
- [9] Y. Zhang, H. Jiang, Y. Tian, H. Ma, X. Zhang, Multigranularity surrogate modeling for evolutionary multiobjective optimization with expensive constraints, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (3) (2024) 2956–2968.
- [10] Z. Song, H. Wang, C. He, Y. Jin, A kriging-assisted two-archive evolutionary algorithm for expensive many-objective optimization, *IEEE Trans. Evol. Comput.* 25 (6) (2021) 1013–1027.
- [11] Z. Song, H. Wang, B. Xue, M. Zhang, Y. Jin, Balancing objective optimization and constraint satisfaction in expensive constrained evolutionary multi-objective optimization, *IEEE Trans. Evol. Comput.* 28 (5) (2024) 1286–1300.
- [12] Y. Tian, J. Hu, C. He, H. Ma, L. Zhang, X. Zhang, A pairwise comparison based surrogate-assisted evolutionary algorithm for expensive multi-objective optimization, *Swarm Evol. Comput.* 80 (2023) 101323.
- [13] Z. Zhang, Y. Wang, G. Sun, T. Pang, K. Tang, Constrained probabilistic Pareto dominance for expensive constrained multiobjective optimization problems, *IEEE Trans. Evol. Comput.* (2024).
- [14] W. Li, R. Mai, Z. Wang, Y. Qiu, B. Xu, Z. Hao, Z. Fan, Surrogate-assisted push and pull search for expensive constrained multi-objective optimization problems, *Swarm Evol. Comput.* 91 (2024) 101728.
- [15] H. Wu, Q. Chen, J. Chen, Y. Jin, J. Ding, X. Zhang, T. Chai, A multi-stage expensive constrained multi-objective optimization algorithm based on ensemble infill criterion, *IEEE Trans. Evol. Comput.* (2024) <http://dx.doi.org/10.1109/TEVC.2024.3400832>.
- [16] S. Liu, Q. Lin, J. Li, K.C. Tan, A survey on learnable evolutionary algorithms for scalable multiobjective optimization, *IEEE Trans. Evol. Comput.* 27 (6) (2023) 1941–1961.
- [17] Z. Fan, W. Li, X. Cai, H. Li, C. Wei, Q. Zhang, K. Deb, E. Goodman, Push and pull search for solving constrained multi-objective optimization problems, *Swarm Evol. Comput.* 44 (2019) 665–679.
- [18] S. Liu, J. Li, Q. Lin, Y. Tian, K.C. Tan, Learning to accelerate evolutionary search for large-scale multiobjective optimization, *IEEE Trans. Evol. Comput.* 27 (1) (2023) 67–81.
- [19] X. Lin, Z. Yang, X. Zhang, Q. Zhang, Pareto set learning for expensive multi-objective optimization, in: *Advances in Neural Information Processing Systems*, Vol. 35, 2022, pp. 19231–19247.
- [20] X. Lin, X. Zhang, Z. Yang, Q. Zhang, Dealing with structure constraints in evolutionary Pareto set learning, *IEEE Trans. Evol. Comput.* (2025) <http://dx.doi.org/10.1109/TEVC.2025.3537986>.
- [21] S. Lai, Z. Zhao, F. Zhu, X. Lin, Q. Zhang, G. Meng, Pareto continual learning: Preference-conditioned learning and adaption for dynamic stability-plasticity trade-off, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 2025, pp. 18008–18016.
- [22] R. Mai, Y. Qiu, W. Li, Z. Wang, B. Xu, W. Chen, J. Zhuang, Y. Li, Z. Fan, Expensive constrained multi-objective evolutionary algorithm with Pareto set learning, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2024, pp. 351–354.
- [23] D.G. Krige, A statistical approach to some mine valuation and allied problems on the Witwatersrand: By DG Krige (Ph.D. thesis), University of the Witwatersrand, 1951.
- [24] J. Tian, Y. Tan, J. Zeng, C. Sun, Y. Jin, Multiobjective infill criterion driven Gaussian process-assisted particle swarm optimization of high-dimensional expensive problems, *IEEE Trans. Evol. Comput.* 23 (3) (2019) 459–472, <http://dx.doi.org/10.1109/TEVC.2018.2869247>.
- [25] Y. Tian, X. Zheng, X. Zhang, Y. Jin, Efficient large-scale multiobjective optimization based on a competitive swarm optimizer, *IEEE Trans. Cybern.* 50 (8) (2020) 3696–3708, <http://dx.doi.org/10.1109/TCYB.2019.2906383>.
- [26] S. Mittal, D.K. Saxena, K. Deb, E.D. Goodman, Enhanced innovized progress operator for evolutionary multi- and many-objective optimization, *IEEE Trans. Evol. Comput.* 26 (5) (2022) 961–975, <http://dx.doi.org/10.1109/TEVC.2021.3131952>.
- [27] Z.-H. Zhan, J.-Y. Li, S. Kwong, J. Zhang, Learning-aided evolution for optimization, *IEEE Trans. Evol. Comput.* 27 (6) (2023) 1794–1808.
- [28] S. Liu, Z. Wang, Q. Lin, J. Li, K.C. Tan, Learning-aided evolutionary search and selection for scaling-up constrained multiobjective optimization, *IEEE Trans. Evol. Comput.* (2024) <http://dx.doi.org/10.1109/TEVC.2024.3380366>.
- [29] E. Zitzler, S. Künzli, Indicator-based selection in multiobjective search, in: *Parallel Problem Solving from Nature - PPSN VIII*, 2004, pp. 832–842.
- [30] K. Li, K. Deb, Q. Zhang, S. Kwong, An evolutionary many-objective optimization algorithm based on dominance and decomposition, *IEEE Trans. Evol. Comput.* 19 (5) (2015) 694–716.
- [31] W. Li, Y. Qiu, Z. Wang, Q. Zhang, R. Mai, B. Xu, Y. Zhang, J. Zhuang, Z. Fan, A constrained multi-objective evolutionary algorithm based on early convergence followed by diversity, in: *2023 5th International Conference on Data-Driven Optimization of Complex Systems, DOCS*, 2023, pp. 1–8.
- [32] O. Ronneberger, P. Fischer, T. Brox, U-Net: Convolutional networks for biomedical image segmentation, in: *Medical Image Computing and Computer-Assisted Intervention - MICCAI 2015*, 2015, pp. 234–241.
- [33] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197.
- [34] D.R. Jones, M. Schonlau, W.J. Welch, Efficient global optimization of expensive black-box functions, *J. Global Optim.* 13 (1998) 455–492.
- [35] Q. Zhang, W. Liu, E. Tsang, B. Virginias, Expensive multiobjective optimization by MOEA/D with Gaussian process model, *IEEE Trans. Evol. Comput.* 14 (3) (2010) 456–474.
- [36] D. Zhan, Y. Cheng, J. Liu, Expected improvement matrix-based infill criteria for expensive multiobjective optimization, *IEEE Trans. Evol. Comput.* 21 (6) (2017) 956–975.
- [37] Z. Fan, W. Li, X. Cai, H. Huang, Y. Fang, Y. You, J. Mo, C. Wei, E. Goodman, An improved epsilon constraint-handling method in MOEA/D for CMOPs with large infeasible regions, *Soft Comput.* 23 (2019) 12491–12510.
- [38] Z. Fan, W. Li, X. Cai, H. Li, C. Wei, Q. Zhang, K. Deb, E. Goodman, Difficulty adjustable and scalable constrained multiobjective test problem toolkit, *Evol. Comput.* 28 (3) (2020) 339–378.
- [39] Z. Ma, Y. Wang, Evolutionary constrained multiobjective optimization: Test suite construction and performance comparisons, *IEEE Trans. Evol. Comput.* 23 (6) (2019) 972–986.
- [40] Z. Wang, S.-L. Shen, D. Chen, W. Li, W. Li, Z. Fan, Multi-objective optimization of the wellbore trajectory considering both geological and engineering factors, *Geoenergy Sci. Eng.* 246 (2025) 213647.
- [41] P. Bosman, D. Thierens, The balance between proximity and diversity in multiobjective evolutionary algorithms, *IEEE Trans. Evol. Comput.* 7 (2) (2003) 174–188.
- [42] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, *IEEE Trans. Evol. Comput.* 3 (4) (1999) 257–271.
- [43] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, *Swarm Evol. Comput.* 1 (1) (2011) 3–18.
- [44] R. Hussein, K. Deb, A generative kriging surrogate model for constrained and unconstrained multi-objective optimization, in: *Proceedings of the Genetic and Evolutionary Computation Conference* 2016, 2016, pp. 573–580.
- [45] H. Wu, Q. Chen, Y. Jin, J. Ding, T. Chai, A surrogate-assisted expensive constrained multi-objective optimization algorithm based on adaptive switching of acquisition functions, *IEEE Trans. Emerg. Top. Comput. Intell.* 8 (2) (2024) 2050–2064.
- [46] Y. Tian, R. Cheng, X. Zhang, Y. Jin, PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum], *IEEE Comput. Intell. Mag.* 12 (4) (2017) 73–87.